
Powerpoint 1: Linux e la shell Bash:

Il prompt dei comandi:

Il prompt dei comandi ha spesso la seguente forma:

```
name@host:workdir$
```

name: il nome di login dell'utente connesso.

host: il nome del computer host del terminale

workdir: il percorso della directory di lavoro.

\$: si è connessi come utente normale mentre **#** se si è connessi come utente root/admin.

Questa forma vale per il prompt di default in molte distribuzioni ma essa può variare a seconda della distribuzione Linux. (Debian, Ubuntu, Kali Linux...)

Esempio:

```
leonardo@pc-ubuntu:~$
```

leonardo → nome utente

pc-ubuntu → nome del computer (hostname)

~ → directory di lavoro (in questo caso la home ~)

\$ → indica che sei un utente normale

Tipi di comandi:

Interni (built-in): Sono funzioni integrate direttamente nella shell.

La loro esecuzione non provoca la creazione di un nuovo task poiché il codice fa parte dell'eseguibile stesso della shell.

Esterni: Sono programmi eseguibili memorizzati sul disco.

La loro esecuzione provoca la creazione di un nuovo task figlio della shell da cui sono avviati.

Script: file testuali in linguaggio shell contenenti sequenze di istruzioni e comandi interpretabili dalla shell.

La loro esecuzione provoca la creazione di una nuova istanza della shell, figlia della shell stessa.

Quando si digita un comando, la shell:

- 1) Verifica se si tratta di un comando interno, nel cui caso lo esegue.
- 2) Altrimenti crea un nuovo processo figlio che esegue il comando.

Metacaratteri ed espressioni:

I metacaratteri sono simboli che la shell interpreta in modo speciale, non come caratteri normali.

Servono per espandere o modificare i comandi, per cercare file, unire comandi, o fare redirezioni di input/output.

Le espressioni sono insiemi di metacaratteri e regole che permettono di scrivere pattern (modelli) per selezionare file o testi.

***** : una qualsiasi stringa di caratteri.

? : un singolo carattere qualsiasi.

[a-h] : un carattere compreso tra 'a' ed 'h' (in valore ASCII)

[aei] : un carattere tra 'a' o 'e' o 'i'.

Esempi:

ls *.txt	mostra tutti i file che finiscono con .txt
ls file?.txt	trova file1.txt, fileA.txt ecc... ma non file10.txt
ls file[a-h].txt	trova i file tra a e h a.txt, filec.txt, fileh.txt
ls file[aei].txt	trova i file: filea.txt, filee.txt, filei.txt

Altri caratteri speciali:

> : redirectione in modalità truncate dello standard output su file (il contenuto del file viene sovrascritto).

>> : redirectione in modalità append dello standard output su file (l'output viene aggiunto in coda al contenuto del file).

< : redirectione dello standard input da file.

& : subito dopo un comando, richiede l'esecuzione di un comando in background (il controllo viene restituito subito all'utente).

; : separatore di comandi all'interno di una stessa riga.

Esempi:

echo "ciao" > output.txt	scrive "ciao" sovrascrivendo l'output di output.txt
echo "nuova riga" >> output.txt	aggiunge all'output del file la riga "nuova riga"
sort < elenco.txt	ordina le righe del file elenco.txt
gedit &	apre gedit e restituisce subito il prompt
cd /etc; ls; pwd	separa i comandi cd /etc, ls e pwd

Per evitare che un metacarattere venga interpretato dalla shell, si usa il backslash ****:

Esempi:

echo *	Stampa tutti i file della directory corrente (es: file1.txt file2.txt)
echo *	Stampa il carattere * trattandolo come un carattere normale

Flag:

Sono opzioni aggiuntive che modificano il comportamento di un comando.

Sono case sensitive e l'ordine in cui si scrivono è generalmente indifferente.

Ogni comando ha propri flag, ed ogni flag può essere:

Semplice: una lettera, in genere preceduta da un trattino.

Esteso: una parola preceduta da due trattini o più parole separate da trattini

Con argomento: flag semplice o esteso seguito da un argomento

Esempi:

ls -R	# mostra i file in modo ricorsivo (Semplice)
ls -all	# mostra anche i file nascosti (Esteso)
ls -f file.txt	# usa "file.txt" come argomento (Con argomento)

Flag letterali: Alcuni comandi hanno flag senza trattino (non molto comuni).

Per usare più flag insieme si può fare in due modi:

Separati: Scrivere i flag in maniera separata: ESEMPIO: ls -l -a -h

Raggruppati: Scrivere i flag in maniera raggruppata: ESEMPIO: ls -lah

Ecco alcuni flag ricorrenti:

--help, -h	#visualizza l'help del comando.
--verbose, -v	#mostra output più dettagliato
--recursive, -r	#esegue l'operazione ricorsivamente all'interno delle subdirectory.
--force, -f	#non chiedere conferma per l'operazione.
--all, -a	#nei comandi che mostrano informazioni su qualcosa è usato per utilizzare insieme tutti i possibili flag (o un sottoinsieme ragionevole di questi).
--version	#mostra la versione del programma (comandi esterni)

N.B) Ogni comando ha i propri flag e le proprie sintassi, vedere sempre il manuale “man” per vedere tutti i flag associati e le loro funzionalità.

Man:

Serve per leggere la documentazione dei comandi, delle funzioni di libreria, dei file speciali ecc...

Mostra un documento formattato chiamato pagina di manuale (*man page*).

Le pagine di manuale sono indicizzate in un database (gestito da **mandb**).

Comandi come “whatis” e “apropos” usano lo stesso database per cercare informazioni.

Struttura tipica di una pagina:

NAME: nome comando e breve descrizione.

SYNOPSIS: sintassi del comando.

DESCRIPTION: descrizione dettagliata.

OPTIONS: lista di tutti i flag con relativa descrizione.

Sintassi generale: **man -[fkKw] [-S sezione] [sezione] [-P paginatore] <comando>**

Flag di man:

-K: cerca una stringa in tutte le pagine di manuale.

-w (o --path): stampa il percorso dei file contenenti le pagine di manuale del comando referenziato.

-P: specifica il paginatore da utilizzare. Default: less

-Sx (o x): mostra le pagine di manuale relative alla sezione x:

- 1: comandi utente (interni ed esterni)
- 2: system call (open, write, fork, ...)
- 3: funzioni di libreria C (printf, scanf, ...)
- 4: file e dispositivi speciali (/dev/null, /dev/stdout, ...)

-k: equivalente ad apropos.

-f: equivalente a whatis.

Esempi:

man -K socket	#cerca la parola "socket" in tutte le pagine di manuale disponibili.
man -w ls	#stampa il percorso del file che contiene la pagina di manuale di ls
man -P cat ls	#mostra la man page di ls direttamente a schermo (senza navigazione con frecce), perché usa cat al posto di less.
man 1 printf	#man page del comando printf (shell, sezione 1)
man -k copy	#elenca tutti i comandi e funzioni che contengono "copy"
man -f ls	#breve descrizione del comando.

N.B) Se ci sono più voci con lo stesso nome, senza specificare la sezione di “man” mostra quella con il numero di sezione più basso.

Info, whatis e apropos:

info: mostra la documentazione in formato Info. (più estesa e navigabile rispetto a man)

Sintassi: **info <comando>**

whatis: ricerca corrispondenze esatte della parola specificata in un database contenente brevi descrizioni dei comandi (campi NAME delle rispettive pagine di manuale).

Sintassi: **whatis <comando>**

Apropos: Simile a whatis, ma mostra anche corrispondenze parziali.

Sintassi: **apropos <comando>**

ESEMPLI:

info ls #Ti apre una documentazione strutturata e navigabile su ls (Duale del comando "man")

INPUT:

whatis man

OUTPUT:

man (1) - an interface to the on-line reference manuals

man (7) - macros to format man pages

INPUT:

apropos grep

OUTPUT:

bzgrep (1) - search possibly bzip2 compressed files

egrep (1) - print lines matching a pattern

fgrep (1) - print lines matching a pattern

git-grep (1) - print lines matching a pattern

grep (1) - print lines matching a pattern

History:

Permette di vedere, richiamare ed eventualmente cancellare lo storico dei comandi eseguiti dall'utente.

Lo storico dipende da variabili d'ambiente: (Definite più avanti)

- **HISTSIZE**: massimo numero di comandi mostrati (o cancellati con flag -c).
- **HISTFILESIZE**: numero di comandi salvati in .bash_history.

Esempi:

history #mostra la cronologia numerata dei comandi digitati.

history -c #elimina la cronologia dei comandi.

Scorciatoie:

!n	#esegue il comando n-esimo nella history.
!-n	#esegue il comando usato n comandi prima.
!!	#esegue l'ultimo comando (equivalente ad !-1).
!stringa	#esegue il comando più recente contenente stringa.

ESEMPI:

INPUT: history

OUTPUT:

```
1 ls
2 cd Documents
3 nano file.txt
4 gcc main.c -o main
5 ./main
```

!3	# riesegue il 3° comando nello storico, cioè: nano file.txt
!-2	# riesegue il comando di 2 posizioni fa ossia cd Documents.
!!	# ripete l'ultimo comando digitato ossia ls.
!ls	# riesegue l'ultimo comando dello storico che iniziava con ls.

Uname:

Serve a visualizzare informazioni relative al kernel, al nome host, all'architettura e al sistema operativo.

Sintassi: `uname [-a] [-snrvmo]`

Flag:

`-a` (--all): mostra tutte le informazioni dei flag successivi (nell'ordine riportato).

`-s` (--kernel-name): nome del kernel.

`-n` (--nodename): hostname della macchina.

`-r` (--kernel-release): release del kernel.

`-v` (--kernel-version): versione del kernel.

`-m` (--machine): architettura della CPU.

`-o` (--operating-system): nome del sistema operativo.

Esempi:

INPUT: `uname -m`

OUTPUT: `x86_64`

INPUT: `uname -r`

OUTPUT: `Linux 4.9.0-14-amd64 x86_64 GNU/Linux`

Alias, unalias, type:

alias: Serve per creare nomi alternativi (scorciatoie) per comandi che hanno priorità su comandi interni ed esterni.

Sintassi: `alias nome=comando`

Esempi: `alias lsa="ls -a"` # "lsa" equivale a digitare "ls -a"

unalias: Rimuove la definizione di un alias.

Sintassi: `unalias [nome | -a]`

Flag: `-a`: elimina tutte le definizioni di alias

Esempio:

`unalias -a` # rimuove tutti gli alias definiti

`unalias lsa` # rimuove solo l'alias "lsa"

type: mostra informazioni riguardanti il tipo di un comando, ossia indica se si tratta di un alias, di un builtin o di un comando esterno.

Sintassi: `type <comando> [comando] ...`

Esempi:

`type ls` # ls is aliased to `ls --color=auto`

`type cd` # cd is a shell builtin

Variabili e ambiente:

Una variabile è identificata da un nome e può contenere un valore (testo, numero, ecc.).

Sintassi: **CREAZIONE:** nome=valore mentre **LETTURA:** \$nome

ESEMPIO:

```
A=34          # assegna il valore ad A pari a 34
$A            #accede al valore di A ossia 34
echo $A       # legge il valore di A ossia 34
```

Le variabili si categorizzano in:

Variabili della shell: Valide solo nella shell attuale e non vengono ereditate dai processi avviati da quella shell.

Esempio: VAR1="ciao"

N.B) Se apri un nuovo terminale, VAR1 non esiste.

Variabili d'ambiente: Valide sia per la shell che per i processi figli eseguiti da quella shell.

Le variabili della shell possono essere trasformate in variabili d'ambiente tramite il comando **export**:

Esempio:

```
VAR2="mondo"      #Creazione variabile della shell
export VAR2       #Trasformazione da variabile shell a variabile ambiente
echo $VAR2        # disponibile nella shell
env | grep VAR2    # disponibile nell'ambiente per i comandi
```

N.B.) Ambiente di un processo UNIX: l'insieme delle variabili d'ambiente ad esso visibili.

Puoi vedere l'ambiente con: **env**

Shell variable → valida solo nella shell.

Environment variable → valida anche nei processi figli (grazie a export).

Alcune variabili d'ambiente comuni:

PATH: percorsi di ricerca dei comandi esterni.

ES:

INPUT: echo \$PATH

OUTPUT: /usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin

HOME: directory home dell'utente loggato.

ES:

INPUT: echo \$HOME

OUTPUT: /home/leonardo

USER: nome dell'utente loggato.

ES:

INPUT: echo \$USER

OUTPUT: leonardo

SHELL: percorso della shell di default.

ES:

INPUT: echo \$SHELL

OUTPUT: /bin/bash

PWD: directory di lavoro corrente.

ES:

INPUT: echo \$PWD

OUTPUT: /home/leonardo/Documents

OLDPWD: directory di lavoro precedente.

ES:

INPUT: echo \$OLDPWD

OUTPUT: /home/leonardo

Alcune variabili della shell comuni:

UID: identificativo numerico dell'utente loggato.

ES:

INPUT: echo \$UID

OUTPUT: 1000

GROUPS: identificativi numerici dei gruppi a cui l'utente appartiene.

ES:

INPUT: echo \$GROUPS

OUTPUT: 1000 27

PPID: PID del processo genitore della shell.

ES:

INPUT: echo \$PPID

OUTPUT: 1234

set, unset, export, printenv:

set: Utilizzato senza argomenti, mostra tutte le variabili della shell e d'ambiente, e le funzioni definite.

Sintassi: **set**

ESEMPIO:

INPUT: set

OUTPUT:

BASH=/bin/bash

BASHOPTS=checkwinsize:cmdhist:...

HISTSIZE=1000

HOME=/home/leonardo

PATH=/usr/local/bin:/usr/bin:/bin

USER=leonardo

unset: Rimuove la definizione di una o più variabili.

Sintassi: `unset <var1> [var2] ...`

Esempio:

```
A=100          #Setto la variabile di shell
echo $A        #Leggo il valore della variabile di shell
unset A        #Rimuovo il valore associato alla variabile di shell
echo $A        # OUTPUT: (vuoto)
```

N.B) Fare `set nome=valore` non crea una variabile di shell ma una stringa

N.B) Per fare l'update basta fare `nome=valore`

export: Esporta una variabile rendendola una variabile d'ambiente.

Sintassi: `export VARIABLE[=valore]`

Esempio:

```
A=200          #Setto la variabile di shell
export A        #Trasformazione da variabile shell a variabile
                ambiente
env | grep A    #Ora A è visibile nell'ambiente
```

printenv: print environment, Mostra tutte le variabili d'ambiente o solo quelle specificate.

Sintassi: `printenv [var1] [var2] ...`

Esempi:

```
printenv        # mostra tutte le variabili d'ambiente
printenv PATH   # mostra solo PATH
```

Il filesystem di Linux:

In Linux, tutto è un file:

Directory → è anch'essa un file, che contiene una lista di directory entry, cioè riferimenti ai file e sottodirectory in essa contenuti.

Periferica di memorizzazione → è vista come un file speciale a blocchi, che supporta operazioni di lettura e scrittura con buffering.

Terminale → è un file speciale a caratteri, che consente l'interazione tramite input/output testuale.

Questo significa che non importa se si tratta di un documento, una cartella, un disco, una stampante o un terminale: per il sistema operativo tutto viene trattato come se fosse un file.

Grazie a questo approccio si possono creare:

1. Creare collegamenti (link simbolici o hard link) tra file, directory e dispositivi.
2. Attribuire permessi uniformi (lettura, scrittura, esecuzione) a file, directory e dispositivi.
3. Leggere e scrivere in modo omogeneo da/su file, directory e dispositivi.

Directory principali e funzione:

/ → directory radice (root), punto di partenza di tutto il filesystem.

/bin → programmi e comandi essenziali, utilizzabili da tutti gli utenti (es. ls, cp, cat).

/sbin → programmi per l'amministrazione del sistema (es. fdisk, iptables), in genere usati da root.

/boot → file necessari all'avvio (bootloader, kernel, initrd).

/dev → file speciali che rappresentano dispositivi hardware e virtuali (es. /dev/sda per dischi, /dev/tty per terminali).

/etc → file di configurazione di sistema e dei servizi.

/home → directory personale di ciascun utente normale (es. /home/mario).

/root → home directory dell'utente amministratore (root).

/tmp → file temporanei, cancellabili al riavvio.

/usr → programmi e librerie utente, documentazione, file di sola lettura (es. /usr/bin, /usr/share).

/var → file variabili che cambiano durante l'uso (log, spool, cache).

/lib → librerie di sistema e moduli del kernel.

/opt → pacchetti software opzionali (installazioni extra, software di terze parti).

/proc → filesystem virtuale con info su processi e sistema (es. /proc/cpuinfo, /proc/1234 per un processo).

/media → punto di mount per dispositivi rimovibili (CD, USB, ecc.).

/mnt → punto di mount temporaneo (montaggi manuali da parte dell'utente/admin).

In sintesi:

- Un utente normale lavora principalmente in /home, usa programmi da /bin e legge configurazioni in /etc.
- Amministratore (root) ha anche accesso a /root, /sbin, /var, /lib, /proc, ecc.
- Dispositivi e processi sono anch'essi accessibili come file dentro /dev e /proc.

N.B) I nomi di file e directory possono contenere qualunque sequenza di caratteri, eccetto / (/ è la directory radice, il punto di partenza di tutto).

La struttura ad albero consente di identificare univocamente un elemento del filesystem mediante il suo percorso assoluto a partire dalla directory radice.

Ogni livello della struttura ad albero è separato da /.

Esempio:

/home/mario/documenti/appunti.txt

Ogni directory contiene due directory speciali (hard link):

`"."`, che punta alla directory stessa.

`".."`, che punta alla directory padre.

Oltre a supportare i percorsi assoluti, la shell consente di utilizzare percorsi relativi, tipicamente riferiti alla directory corrente. Un percorso relativo non inizia con /.

Esempi:

<code>./file.txt</code>	<code># file nella directory corrente</code>
<code>../foto/img.png</code>	<code># file dentro la directory padre</code>
<code>documenti/appunti.txt</code>	<code># percorso relativo senza ./ iniziale</code>

Esempi:

Percorso assoluto a un file:	<code>/home/mario/appunti.txt</code>
Percorso relativo allo stesso file:	<code>appunti.txt</code> oppure <code>./appunti.txt</code>
Percorso relativo da una sottodirectory documenti:	<code>../appunti.txt</code>

Permessi di accesso:

Ad ogni file o directory possono essere assegnati tre tipi di permesso:

1. Lettura (read → r)

File: permette di leggerne il contenuto.

Directory: permette di elencarne i file (ls).

2. Scrittura (write → w)

File: permette di modificarne il contenuto.

Directory: permette di creare, rinominare o cancellare file all'interno.

3. Esecuzione (execute → x)

File: permette di eseguire il file come programma/script.

Directory: permette di entrarci (cd) e accedere ai file conoscendone il nome.

Quando un file viene creato, esso viene collegato all'utente che lo ha creato e al gruppo principale di cui fa parte.

I permessi sono assegnati a tre categorie distinte:

1. **User (u)** → il proprietario del file. (interno più ristretto)
2. **Group (g)** → gli utenti che appartengono al gruppo del file. (cerchio più largo che include user).
3. **Other (o)** → tutti gli altri utenti del sistema. (cerchio più ampio che include tutti).

Access Control List: (ACL)

Ogni file o directory è associato a una **stringa di 10 caratteri**, che codifica tipo e permessi.

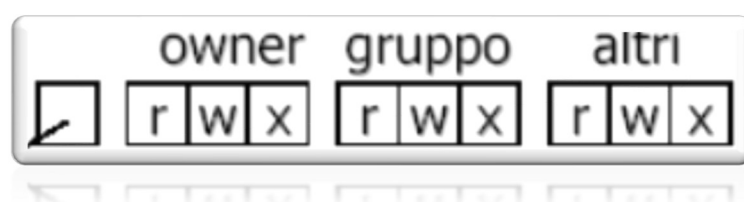
I caratteri di una ACL sono suddivisi in quattro gruppi: **[tipo] [utente] [gruppo] [altri]**

l: tipo di file.

u: permessi concessi al proprietario del file.

g: permessi concessi agli utenti membri del gruppo.

o: permessi agli altri utenti, non compresi nei precedenti insieme.



Tipi di file:

1. **-** → file ordinario: Contiene dati o programmi.
2. **d** → directory: Contiene altre directory o file.
3. **b** → file speciale a blocchi: Dispositivi di memorizzazione (es. hard disk, CD/DVD). Accesso bufferizzato a blocchi di dimensione fissa.
4. **c** → file speciale a caratteri: Dispositivi che trasferiscono dati un carattere alla volta, senza bufferizzazione. (terminali, porte seriali, tastiere, schede audio)

Significato dei permessi:

1. File ordinari

- r (read) → leggere il contenuto del file.
- w (write) → modificare il contenuto del file.
- x (execute) → eseguire il file come programma/script.

2. Directory

- r (read) → permette di elencare i file contenuti (ls).
- w (write) → permette di modificare la directory (creare,eliminare, rinominare file)
- x (execute) → consente di "entrare" nella directory (cd) e accedere ai file se ne conosce il nome (anche senza permesso di lettura).

Esempio: una directory con permessi --x consente di entrare (cd dir) e aprire un file se conosco il nome, ma non di elencarne il contenuto.

3. File speciali

- r (read) → leggere dati dal device (input).
- w (write) → scrivere dati sul device (output).
- x (execute) → Non configurato

N.B) Nell'ACL, l'assenza di un permesso è identificata da un trattino (-).

Esempio: (-File Ordinario)

- rw- r-- ---

Proprietario: può leggere e scrivere.

Gruppo: può leggere.

Altri: nessun permesso.

Al momento della creazione, i permessi di default sono:

- **rw- r-- r—** : per i file ordinari che possono essere letti e modificati dal proprietario, e solo letti da tutti gli altri.
- **d rwx r-x r-x**: per le directory che possono essere lette, modificate e scansionate dal proprietario, e solo lette e scansionate dagli altri.

Chmod:

chmod => change mode: Modifica i permessi (r, w, x) di file o directory.

Può essere usato solo dal proprietario del file o dall'utente root!

Sintassi: **chmod [-R] <mode> <file> [file] ...**

<mode> può essere in formato simbolico (u, g, o, a con +, -, =) o ottale (notazione numerica basata su 3 cifre)

ESEMPIO:

chmod u+x script.sh → aggiunge esecuzione all'utente

chmod 755 script.sh → rwx r-x r-x

Flag: **-R** → applica la modifica in modo ricorsivo a tutte le sottodirectory e file.

Permessi - forma simbolica:

La forma simbolica combina codici utente, codici di permesso e operatori:

Codici utente:

u → utente proprietario (user)

g → gruppo proprietario (group)

o → altri utenti (others)

a → tutti (equivalente a ugo)

Codici di permesso:

r → lettura (read)

w → scrittura (write)

x → esecuzione (execute)

Operatori:

+ → aggiunge un permesso

- → rimuove un permesso

= → assegna esattamente i permessi specificati (togliendo gli altri)

Esempi:

1. `chmod g+w file.txt`

→ aggiunge scrittura al gruppo

→ se prima era `rw-r--r--` diventa `rw-rw-r--`.

2. `chmod ugo+rw file.txt` (o `a+rw file.txt`)

→ assegna lettura e scrittura a tutti (utente, gruppo, altri).

3. `chmod o-x file.txt`

→ rimuove esecuzione agli altri utenti.

4. `chmod a=r documento.txt`

→ tutti possono solo leggere (qualsiasi permesso precedente viene eliminato).

Permessi - forma ottale:

I permessi sono rappresentati con tre cifre ottali (da 0 a 7).

Ogni cifra rappresenta i permessi di:

- prima cifra → utente (user)
- seconda cifra → gruppo (group)
- terza cifra → altri (others)

r	w	x	Cifra in forma ottale	significato
0	0	0	0	Nessun permesso
0	0	1	1 (2^0)	Esecuzione solo
0	1	0	2 (2^1)	Scrittura solo
0	1	1	3 (2^1+2^0)	Scrittura ed esecuzione
1	0	0	4 (2^2)	Lettura solo
1	0	1	5 (2^2+2^0)	Lettura ed esecuzione
1	1	0	6 (2^2+2^1)	Lettura e scrittura
1	1	1	7 ($2^2+2^1+2^0$)	Lettura scrittura esecuzione

ESEMPI:

1. `chmod 644 file.txt` → `rw-r--r--`

Utente: lettura + scrittura

Gruppo: sola lettura

Altri: sola lettura

2. `chmod 755 script.sh` → `rwxr-xr-x`

Utente: lettura + scrittura + esecuzione

Gruppo: lettura + esecuzione

Altri: lettura + esecuzione

3. `chmod 700 config` → `rwx-----`

Solo l'utente proprietario ha pieno accesso

Chown:

`chown` = change owner => Permette di modificare:

- utente proprietario di un file/directory
- gruppo proprietario del file/directory

N.B) Può essere usato solo da root (o tramite `sudo` che da temporaneamente i permessi di root)

Sintassi: `chown [-R] [utente][:[gruppo]] <file> ...`

[utente] → nuovo proprietario

[gruppo] → nuovo gruppo

-R → applica ricorsivamente a tutte le sottodirectory e file contenuti

ESEMPI:

1. `sudo chown utente1 file.txt` → ora il file appartiene a utente1.

2. `sudo chown -R utente1:gruppo1 cartella/` → tutta la directory cartella e i file al suo interno appartengono a utente1 e al gruppo gruppo1.

3. `sudo chown :gruppo1 file.txt` → l'utente resta invariato, ma il gruppo diventa gruppo1.

Chgrp:

chgrp = change group => Modifica solo il gruppo proprietario di uno o più file/directory.

N.B) Può essere eseguito solo dall'utente root o tramite sudo.

Sintassi: `chgrp [-R] <gruppo> <file> [file] ...`

<gruppo> = nuovo gruppo proprietario

-R = applica ricorsivamente a tutte le sottodirectory e ai file contenuti

ESEMPLI:

1. `sudo chgrp gruppo1 file.txt` → ora file.txt appartiene al gruppo gruppo1.
2. `sudo chgrp -R studenti documenti/` → la directory documenti e tutti i file al suo interno appartengono al gruppo studenti.

Differenza con chown:

- `chown` → cambia utente proprietario e opzionalmente anche il gruppo.
- `chgrp` → cambia solo il gruppo proprietario.

Infatti, questo comando è usato quando si vuole lasciare invariato l'utente, ma assegnare i file a un altro gruppo (es. da staff a students).

Pwd:

pwd = print working directory => Mostra sullo standard output il percorso assoluto della directory di lavoro corrente.

Sintassi: `pwd`

ESEMPIO: `pwd` => /home/user

Cd:

cd = change directory => Modifica la directory di lavoro corrente (quella che puoi vedere con pwd).

Sintassi: `cd [percorso]`

[percorso] può essere assoluto (inizia con /) o relativo.

Esempi:

- **cd**
→ porta l'utente nella home directory (/home/nomeutente).
→ Equivalente a cd ~.
- **cd -**
→ torna alla directory precedente.
(utile per passare avanti e indietro tra due cartelle).
- **cd .**
→ nessun effetto (rimane nella directory corrente).
- **cd ..**
→ porta nella directory padre di quella corrente.

Esempio pratico:

```
pwd                                #Mostro il percorso corrente
/home/user/documenti
```

```
cd ..                             #Torno nella directory padre
pwd
/home/user
```

```
cd -                               #Torno nella directory precedente
pwd
/home/user/documenti
```

Ls:

ls = list => Elenca i file e le directory presenti in un percorso.

Sintassi: **ls [flag] [percorso]**

Flag principali:

-a → mostra tutti i file, inclusi quelli nascosti (. e .., e i file che iniziano con .).

-F → aggiunge un simbolo per indicare il tipo:

***** file eseguibile

/ directory

@ link simbolico

-r → inverte l'ordine di visualizzazione.

-R → mostra ricorsivamente anche il contenuto delle sottodirectory.

-t → ordina per data di ultima modifica (dal più recente).

-u → ordina per data di ultimo accesso.

-i → mostra l'inode number dei file (identificatore univoco del file nel filesystem).

-1 → stampa i file uno per riga.

-l → mostra dettagli (ACL, link, utente, gruppo, dimensione, data, nome).

[ACL] [# hard link] [utente] [gruppo] [dimensione (b)] [ultima modifica] [nome]

ESEMPIO:

INPUT: `ls -l`

OUTPUT:

`-rw-r--r-- 1 mario studenti 512 Aug 17 20:00 appunti.txt`

`drwxr-xr-x 2 mario studenti 4096 Aug 17 20:05 document`

touch:

Aggiorna i timestamp (data e ora) di accesso e modifica di un file.

N.B) Se il file non esiste, lo crea come file vuoto.

Sintassi: **touch [flag] <percorso>**

Flag principali:

-a → aggiorna solo il timestamp di accesso.

-m → aggiorna solo il timestamp di modifica.

-c → non crea il file se non esiste.

Esempi pratici:

1. Creare un file vuoto: `touch nuovo.txt`
2. Aggiornare i timestamp di un file: `touch myfile`
3. Aggiornare solo il timestamp di accesso senza creare il file: `touch -ca myfile`

Mkdir:

mkdir = make directory => Crea una nuova directory in un percorso specificato.

Sintassi: `mkdir [flag] <percorso> ...`

Flag principali:

`-m` <ACL> → permette di specificare subito i permessi della directory (Access Control List).

Esempio: `mkdir -m 755 nuova_dir`

`-p` → non mostra errore se la directory esiste già e crea automaticamente eventuali directory padre mancanti.

Esempio: `mkdir -p progetto/src/include`

`-v` → modalità verbosa, stampa messaggi di conferma.

Esempio: `mkdir -v nuova_dir`

Rmdir:

rmdir = remove directory => Cancella una directory solo se vuota.

N.B) Se la directory contiene file o altre sottodirectory → non funziona (si usa `rm -r`).

Sintassi: `rmdir [flag] <percorso>`

Flag principali:

`--ignore-fail-on-non-empty` → non mostra errori se la directory non è vuota (semplicemente la ignora).

`-v` → modalità *verbosa*, mostra messaggi per ogni directory rimossa.

`-r` → rimuove directory anche se contengono file (pericoloso se usato senza attenzione).

Esempi pratici:

1. Cancellare una directory vuota: `rmdir cartella_vuota`
2. Ignorare errori se non vuota: `rmdir --ignore-fail-on-non-empty cartella`
→ nessun messaggio di errore, ma cartella rimane.
3. Cancellare più directory vuote: `rmdir dir1 dir2 dir3`

Cp:

cp => Copia file e directory

Copia uno o più file/directory in un percorso di destinazione.

Se la destinazione è una directory, I file vengono copiati all'interno mantenendo lo stesso nome.

Se la destinazione è un file, Il file sorgente viene copiato con un nuovo nome.
(Ridenominazione file)

Sintassi: `cp [flag] <percorso_origine> ... <percorso_destinazione>`

N.B) Puoi usare sia percorsi relativi se ti trovi nella cartella di interesse, sia percorsi assoluti e puoi anche copiare anche più file insieme.

Flag principali:

`-n` → non sovrascrivere file esistenti (di default i file vengono sovrascritti).

`-i` → chiede conferma prima di sovrascrivere (interattivo).

`-f` → forza la copia: se un file esistente non può essere aperto in scrittura, viene rimosso e ricreato.

`-r` → copia una directory ricorsivamente (obbligatorio per copiare directory).

`-v` → modalità verbosa, mostra le operazioni eseguite.

ESEMPI:

1. `cp documento.txt backup/` => Copia documento.txt dentro la cartella backup/ mantenendo lo stesso nome.
2. `cp documento.txt documento_backup.txt` => Copia documento.txt in un nuovo file chiamato documento_backup.txt.
3. `cp -n documento.txt backup/` => Copia documento.txt in backup/ solo se non esiste già.
4. `cp -i documento.txt backup/` => Se documento.txt esiste già in backup/, il terminale chiede conferma: overwrite 'backup/documento.txt'? (y/n)
5. `cp -f documento.txt backup/` => Se non riesce a scrivere su backup/documento.txt, lo elimina e lo ricrea.

6. `cp -r progetto/ backup/` => Copia tutta la cartella progetto/ (con i suoi file e sottocartelle) dentro backup/.
7. `cp -rv progetto/ backup/` => Stampa ogni file man mano che viene copiato, ad esempio:
`'progetto/file1.txt' -> 'backup/file1.txt'`
`'progetto/subdir/file2.txt' -> 'backup/subdir/file2.txt'`

Mv:

`mv` => Move: Sposta file e directory

N.B) Non fa la copia, ma toglie dal percorso origine e mette in quello di destinazione.

Rinominare un file è un caso speciale di spostamento: sposti il file nella stessa directory ma con un nuovo nome.

Di default sovrascrive i file esistenti senza chiedere conferma (a meno di usare flag).

Sintassi: `mv [flag] <percorso_origine> ... <percorso_destinazione>`

N.B) Puoi muovere più file specificandoli uno dopo l'altro.

Flag principali:

`-i` → chiede conferma in caso di sovrascrittura.

`-f` → forza la sovrascrittura (anche se non hai permessi di scrittura sul file di destinazione).

`-v` → modalità verbosa, mostra cosa viene spostato/rinominato.

ESEMPI:

1. `mv documento.txt archivio/` => Sposta documento.txt dentro archivio/.
2. `mv documento.txt documento_vecchio.txt` => Stesso percorso, cambia solo il nome.
3. `mv file1.txt file2.txt backup/` => Sposta file1.txt e file2.txt dentro backup/.
4. `mv -i documento.txt archivio/` => Se documento.txt esiste già in archivio/, ti chiede: `overwrite 'archivio/documento.txt'? (y/n)`
5. `mv -v documento.txt archivio/` => Sposta documento.txt dentro archivio/ mostrando in output: `'documento.txt' -> 'archivio/documento.txt'`

rm:

rm => Remove: Rimuove file e directory.

N.B) Di default le directory possono essere eliminate solo se vuote, per cancellare directory con contenuto, serve il flag -r.

N.B) Se non hai i privilegi di scrittura, rm chiede conferma (a meno che usi -f).

Sintassi: `rm [flag] <percorso> ...`

Flag principali:

`-r` → elimina directory ricorsivamente (anche se contengono file/sottocartelle).

`-i` → chiede conferma per ogni file prima di eliminarlo.

`-f` → forza la rimozione: ignora file inesistenti e non chiede mai conferma.

`-v` → modalità verbosa, mostra i file man mano che vengono eliminati.

ESEMPLI:

1. `rm documento.txt` => Rimuove documento.txt.
2. `rm file1.txt file2.txt file3.txt` => Rimuove tutti e tre i file.
3. `rm nomecartella/` => Funziona solo se la directory è vuota.
4. `rm -r progetto/` => Cancella la cartella progetto/ e tutto ciò che contiene.
5. `rm -i documento.txt` => Prima di rimuovere documento.txt. ti chiede:
rm: remove regular file 'documento.txt'? (y/n)
6. `rm -rf progetto/` => Cancella progetto/ e tutto dentro, senza chiedere nulla.
7. `rm -rv progetto/` => Mostra i file/directory eliminati:
removed 'progetto/file1.txt'
removed directory 'progetto/subdir'
removed directory 'progetto'

file, stat:

file => Determina il tipo di un file: Non guarda solo l'estensione, ma il contenuto interno per capire se è testo, immagine, eseguibile, ecc.

Sintassi: **file [flag] <percorso>**

Esempio:

INPUT: file star_wars.txt

OUTPUT: star_wars.txt: ASCII text

stat => Mostra informazioni dettagliate su un file, ossia:

1. dimensione,
2. numero di blocchi occupati,
3. tipo (regular file, directory, link, ecc.),
4. permessi,
5. proprietario e gruppo,
6. date di accesso, modifica e cambiamento dei metadata.

Sintassi: **stat [flag] <percorso>**

Esempio:

INPUT: stat star_wars.txt

OUTPUT:

File: star_wars.txt

Size: 276 Blocks: 8 IO Block: 4096 regular file

Device: 801h/2049d Inode: 918269 Links: 1

Access: (0644/-rw-r--r--) Uid: (1000/ ivano) Gid: (1000/ ivano)

Access: 2020-12-08 16:07:54.647659451 +0100

Modify: 2019-10-28 18:31:35.33151954 +0100

Change: 2019-10-29 15:43:36.778988058 +0100

Differenze rapide:

file → ti dice che tipo di file è (testo, immagine, binario, ecc.).

stat → ti dà tutti i dettagli tecnici (dimensioni, permessi, date, inode, ecc.).

Link:

Inode e file system:

Inode e file system: In UNIX/Linux ogni file/directory/link è descritto da una struttura dati chiamata *inode*.

L'inode contiene:

- attributi: permessi, proprietario, tempi di accesso/modifica, numero di link...
- puntatori ai data block, dove sta il contenuto reale del file.

Ogni file ha un inode number univoco nel filesystem.

Le directory contengono directory entry ossia le coppie (inode number, filename) dei file.

Tipi di link:

Esistono due tipi di link in Linux:

- Hard link:

È un nuovo nome che punta allo stesso inode di un file esistente.

File originale e hard link sono indistinguibili (hanno lo stesso inode e quindi stesso contenuto).

Se uno dei due viene eliminato, l'altro continua a funzionare finché il link count > 0.

- Soft link (o symbolic link / symlink)

È un file speciale che contiene il percorso di un altro file.

Ha un inode diverso e se il file originale viene eliminato → il symlink diventa "rotto"

Equivalente alle scorciatoie in Windows.

Link e directory:

Una directory ha almeno 2 hard link:

1. uno dal padre (nome_cartella/),
2. uno da sé stessa (.).

Ogni nuova sotto-directory crea automaticamente un hard link al padre (..).

I soft link non modificano l'inode del file di destinazione.

Gli hard link modificano il link count dell'inode:

1. creazione → link count +1
2. rimozione → link count -1
3. se link count = 0 → inode e blocchi dati vengono deallocati.

Limitazioni:

- Hard link:

Non possono puntare a file su filesystem diversi (es. partizioni diverse).

Nelle implementazioni moderne, non si possono creare hard link a directory (per evitare loop ricorsivi).

- Soft link:

Possono puntare a file anche su filesystem diversi.

Possono puntare a directory senza problemi.

Caratteristica	Hard Link	Soft Link
Inode	Stesso inode dell'originale	Inode diverso
Contenuto	Identico, condividono i data block	Contiene il percorso del file originale
Se elimini l'originale	Il link continua a funzionare	Il link diventa rotto
File system	Solo stesso filesystem	Può attraversare filesystem diversi
Directory	Di solito non permessi	Consentiti
Riconoscibile	Indistinguibile dall'originale	Si riconosce facilmente (→ ls -l)

ln:

ln => link: Serve per creare collegamenti (link) tra file.

N.B) Di default crea un hard link ma con l'opzione -s crea invece un soft link

Sintassi: `ln [flags] <percorso_origine> ... <percorso_destinazione>`

Flag principali:

-s → crea un soft link invece che un hard link.

ESEMPI:

1. `ln myfile1 myfile2` => Crea un hard link chiamato myfile2 che punta allo stesso inode di myfile1. (myfile1 e myfile2 sono indistinguibili).
2. `ln -s myfile1 myfile` => Crea un softlink in myfile2 che contiene il percorso di myfile1.
N.B) Se elimini myfile1, myfile2 diventa un link rotto.
3. `ln myfile1 myfile2 mydir/` => Crea hard link a myfile1 e myfile2 dentro mydir/.

Powerpoint 3: Stream ed espressioni regolari:

Stream e Redirezione:

Ogni programma ha 3 flussi di I/O, identificati da un file descriptor:

0 → stdin (input standard) → da tastiera (default).

1 → stdout (output standard) → a terminale (default).

2 → stderr (errori standard) → a terminale (default).

Redirezione: `comando [file_descriptor] <operatore> file`

Operatori principali:

< → redirezione input (stdin).

> → redirezione output (stdout) in truncate mode (sovrascrive file).

>> → redirezione output in append mode (aggiunge al file senza cancellare).

>& → concatenazione di più file descriptor.

N.B) Se non specifichi il file descriptor:

> si riferisce a stdout (1)

< si riferisce a stdin (0)

ESEMPLI:

1. `ls > myfile.txt` => Scrive l'output di ls in myfile.txt (sovrascrive).
2. `ls pippo 2> errori.txt` => Se pippo non esiste, l'errore va in errori.txt.
3. `ls pippo 2>> errori.txt` => Gli errori vengono aggiunti a errori.txt (non sovrascritti).
4. `ls pippo > out.txt 2>&1` => Output e errori vanno entrambi in out.txt.
5. `sort < numeri.txt` => Legge i dati da numeri.txt invece che da tastiera.

6. `cmd < myin.txt > myout.txt 2> myerr.txt` => input da myin.txt, output in myout.txt e error in myerr.txt.
7. `cmd < myin.txt >> myout.txt 2>&1` => input da myin.txt mentre output ed error insieme in myout.txt (append mode).

Sequenze di comandi:

In Linux puoi concatenare comandi in diversi modi:

Punto e virgola: ;

SINTASSI: `comando1 ; comando2`

Esegue prima comando1, poi comando2, indipendentemente dal successo o errore del primo.

Operatore && (AND logico): &&

SINTASSI: `comando1 && comando2`

Esegue comando2 solo se comando1 ha avuto successo.

ESEMPIO: `mkdir backup && cp file.txt backup/` #se mkdir fallisce, non copia il file.

Operatore || (OR logico): ||

SINTASSI: `comando1 || comando2`

Esegue comando2 solo se comando1 fallisce.

ESEMPIO: `ls pippo || echo "Cartella non trovata"` #O mostra pippo o mostra echo

Operatore | Pipeline: |

SINTASSI: `comando1 | comando2`

Manda l'output del comando1 come input di comando2 e poi esegue comando2

ESEMPIO: `ls pippo | sort` #Mostra pippo ordinato tramite sort

Utilità delle pipeline:

Permettono di collegare più comandi in sequenza, creando vere e proprie catene di elaborazione (pipeline): l'output di un comando diventa input del successivo.

echo:

Serve per stampare un messaggio su standard output.

Di default lo mostra a schermo, ma si può redirigere verso file o altri comandi.

Sintassi: **echo [stringa]**

N.B) Se non scrivi nulla dopo echo, stampa una **riga vuota**.

ESEMPLI:

1. echo "Hello, world!" #Stampa "Hello, world!"
2. echo ~ #Stampa la home directory dell'utente
3. echo "Hello, world!" > myfile.txt #Scrive "Hello, world!" in myfile.txt.

cat:

cat => concatenate: Serve a stampare o concatenare il contenuto di uno o più file, mostrandolo su stdout (il terminale di default).

Spesso usato insieme alla redirectione > o >> per scrivere su file.

Sintassi: **cat [flags] [percorso] ...**

Flag principali:

-n → numera le righe dell'output.

ESEMPLI:

1. cat myfile1.txt myfile2.txt => Stampa il contenuto di myfile1.txt e myfile2.txt concatenati sul terminale.
2. cat myfile1.txt => Mostra a schermo tutto il contenuto di myfile1.txt.
3. cat myfile1.txt myfile2.txt > myfile3.txt => Unisce il contenuto di myfile1.txt e myfile2.txt e lo scrive in myfile3.txt.

N.B) Se myfile3.txt esiste viene sovrascritto mentre se non esiste viene creato automaticamente.

4. `cat myfile1.txt >> myfile2.txt` => Aggiunge il contenuto di myfile1.txt alla fine di myfile2.txt (non sovrascrive).
5. `cat -n myfile.txt` => Stampa myfile.txt numerando ogni riga.

more, less:

Sono programmi di paginazione che servono a visualizzare un file di grandi dimensioni a pagine invece che tutto insieme (come fa cat).

more → versione più semplice che permette solo di andare avanti.

less → più avanzato, permette di scorrere avanti e indietro, cercare stringhe, muoversi facilmente.

Per questo, oggi less ha sostituito more come standard.

Sintassi:

`more <percorso>`

`less <percorso>`

ESEMPI:

1. `less myfile.txt` => Visualizza myfile.txt nel paginatore.
2. `ls -la | less` => Mostra l'output di `ls -la` paginato (Utile se l'elenco è molto lungo).

Comandi utili dentro less

- Freccia su/giù → scorrere riga per riga.
- Spazio → avanti di una pagina.
- `b` → indietro di una pagina.
- `/testo` → cerca "testo" nel file.
- `n` → vai alla prossima occorrenza della ricerca.
- `q` → esci.

sort:

Ordina le righe di uno o più file.

Di default si usa l'ordine lessicografico (alfabetico + numerico trattati come stringhe).

Sintassi: `sort [flags] <percorso> ...`

Flag principali:

`-c` → controlla se il file è già ordinato; se no, stampa la prima riga fuori posto.

`-n` → ordina numericamente (i numeri vengono trattati come valori, non stringhe).

`-f` → ignora maiuscole/minuscole (case-insensitive).

`-r` → inverte l'ordinamento (ordine decrescente).

`-k N` → ordina secondo il campo N-esimo (di default i campi sono separati da spazi o tab).

`-t "sep"` → specifica un separatore di campo (es. `-t,` per CSV).

ESEMPI:

1. `sort nomi.txt` => Ordina le righe di nomi.txt in ordine alfabetico.
2. `sort -n numeri.txt` => Ordina i numeri come valori, non come stringhe.
3. `sort -r nomi.txt` => Ordina dalla Z alla A.
4. `sort -f nomi.txt` => Ordina ignorando maiuscole/minuscole (A = a).

5. `sort -k 2 -n dati.txt` => Ordina in base al secondo campo, trattandolo come numero:

Input:	Output:
mario 30	anna 25
anna 25	luca 28
luca 28	mario 30

6. `sort -t, -k 2 -n dati.txt` => Usa , come separatore e ordina per secondo campo.

Input:	Output:
mario, 30	anna, 25
anna, 25	luca, 28
luca, 28	mario, 30

head:

Mostra le prime righe o i primi byte di un file (Di default mostra le prime 10 righe)

Sintassi: `head [flags] <percorso>`

Flag principali:

`-c N` → mostra i primi N byte.

`-c -N` → mostra tutti i byte esclusi gli ultimi N.

`-n N` → mostra le prime N righe (equivalente a `-N`).

`-n -N` → mostra tutte le righe escluse le ultime N.

ESEMPI:

1. `head file.txt` => Prime 10 righe di un file (default)
2. `head -n 5 file.txt` => Prime 5 righe
3. `head -c 20 file.txt` => Prime 20 byte
4. `head -n -3 file.txt` => Tutte le righe tranne le ultime 3
5. `head -c -100 file.txt` => Tutti i byte tranne gli ultimi 100

tail:

Mostra le ultime righe o gli ultimi byte di un file (Default: mostra le ultime 10 righe).

Sintassi: `tail [flags] <percorso>`

Flag principali:

`-c N` → mostra gli ultimi N byte.

`-c +N` → mostra i byte a partire dal numero N fino alla fine.

`-n N` → mostra le ultime N righe (equivalente a `-N`).

`-n +N` → mostra le righe a partire dalla N-esima fino alla fine.

ESEMPI:

1. `tail file.txt` => Ultime 10 righe di un file (default)
2. `tail -n 5 file.txt` => Ultime 5 righe
3. `tail -c 20 file.txt` => Ultimi 20 byte
4. `tail -n +3 file.txt` => Mostra dalla terza riga fino alla fine del file.
5. `tail -c +15 file.txt` => Mostrare dal 15° byte in poi

N.B) Una riga è l'insieme di caratteri fino al carattere di a capo (`\n`).

Quindi `-n` lavora "per testo" → conta quante righe ci sono, indipendentemente dalla lunghezza.

N.B) Un byte è la quantità di dati elementare (1 carattere $\approx$ 1 byte in ASCII).

Quindi `-c` lavora "per quantità di dati" → prende esattamente quel numero di byte dal file, senza preoccuparsi delle righe.

N.B) In codifica ASCII Caratteri = Byte ma non è sempre così (Unicode)

uniq:

Mostra o elimina le righe ripetute in un file o in un flusso di testo.

Default: rimuove le righe duplicate adiacenti, mostrando solo una copia.

N.B) Le righe duplicate devono essere adiacenti → spesso si usa prima `sort`.

Sintassi: `uniq [flags] <percorso>`

Flag principali:

`-d` → mostra solo le linee duplicate.

`-u` → mostra solo le linee uniche (non ripetute).

`-c` → mostra il conteggio di occorrenze per ogni riga.

`-i` → confronto case-insensitive (ignora maiuscole/minuscole).

ESEMPLI:

1. `uniq nomi.txt` => Rimuovere duplicati adiacenti

INPUT:	OUTPUT:
mario	mario
anna	anna
mario	mario
anna	anna
anna	luca
luca	

2. `sort nomi.txt | uniq` => Rimuove tutti i duplicati

INPUT:	OUTPUT:
mario	mario
anna	anna
mario	luca
anna	
anna	
luca	

3. `uniq -c nomi.txt` => Con conteggio

OUTPUT:

1 mario
2 anna
1 mario
2 anna
1 luca

4. `uniq -d nomi.txt` => Mostrare solo righe duplicate adiacenti

OUTPUT:

anna

5. `uniq -u nomi.txt` => Mostrare solo righe uniche adiacenti

OUTPUT:

mario
mario
luca

wc:

Conta righe, parole, caratteri e byte per ciascun file.

Default: mostra righe, parole e byte.

Il formato è: [righe] [parole] [caratteri] [byte] percorso

Sintassi: **wc [flags] <percorso>**

Flag principali:

-l → conta le righe.

-w → conta le parole.

-m → conta i caratteri.

-c → conta i byte.

N.B) In ASCII, 1 carattere = 1 byte ma non è sempre così (Unicode...)

ESEMPLI:

Supponiamo che file.txt contenga:

ciao mondo

linux è bello

1. **wc file.txt** => Contare righe, parole e byte (default)

OUTPUT: 2 4 23 file.txt

2. **wc -l file.txt** => Contare solo le righe

OUTPUT: 2 file.txt

3. **wc -w file.txt** => Contare solo le parole

OUTPUT: 4 file.txt

4. **wc -m file.txt** => Contare solo i caratteri

OUTPUT: 23 file.txt

5. `wc -c file.txt` => Contare solo i byte
OUTPUT: 23 file.txt

cut:

Estrae porzioni di ogni riga di un file o di un input.

Può estrarre byte specifici, caratteri specifici o campi delimitati da un separatore (di default: TAB).

Sintassi: `cut [flags] <percorso>`

Flag principali:

`-b <range>` → estrae i byte indicati.

`-c <range>` → estrae i caratteri indicati.

`-f <range>` → estrae i campi indicati.

`-d <sep>` → specifica il separatore di campo (default: TAB).

Un `<range>` può assumere varie forme:

`N` → estrae l'elemento N.

`N-M` → estrae gli elementi da N a M.

`N-` → estrae gli elementi da N fino alla fine.

`-M` → estrae gli elementi dall'inizio fino a M.

ESEMPLI:

Supponiamo che `dati.txt` contenga:

mario,rossi,30

anna,verdi,25

luca,neri,28

1. `cut -c 1-4 dati.txt` => Estrarre i primi 4 caratteri

Output:

mari

anna

luca

2. `cut -d, -f 2 dati.txt` => Estrarre il secondo campo (con separatore ,)

Output:

rossi

verdi

neri

3. `cut -d, -f 1,3 dati.txt` => Estrarre campi 1 e 3

Output:

mario,30

anna,25

luca,28

4. `cut -d, -f2- dati.txt` => Estrarre dal secondo campo fino alla fine

Output:

rossi,30

verdi,25

neri,28

diff:

Confronta due file o due directory, mostrando le differenze.

Utile per confrontare file di configurazione, script o directory di progetto.

Esiste una variante colordiff che evidenzia le differenze con colori.

Sintassi: `diff [flags] <percorso_1> <percorso_2>`

Flag principali:

`-r` → se i percorsi sono directory, confronta ricorsivamente anche le sottodirectory.

`--brief` → indica solo se ci sono differenze, senza mostrarle.

`--ignore-case` → ignora differenze tra maiuscole e minuscole.

ESEMPI:

Supponiamo che File f1.txt contenga:

File f1.txt:

ciao

mondo

Supponiamo che File f2.txt contenga:

File f2.txt:

ciao

Linux

1. `diff f1.txt f2.txt`

Output:

2c2

< mondo

> Linux

Significato: alla riga 2 i contenuti sono diversi (mondo vs Linux).

2. `diff -r dir1 dir2` => Mostra file aggiunti, rimossi o modificati tra dir1 e dir2.
3. `diff --brief f1.txt f2.txt` => indica solo se ci sono differenze, senza mostrarle.
Output: Files f1.txt and f2.txt differ
4. `diff --ignore-case f1.txt f2.txt` => Tratta Linux e linux come uguali.

Espressioni regolari:

Un'espressione regolare (regex) è un pattern che serve a identificare un insieme di stringhe.

Sintassi spiegata con esempi:

1. `a` → una singola occorrenza del carattere a.

Esempio: "cane" contiene a.

2. `[abcd]` → un singolo carattere tra a, b, c, d.

Esempio: "b", "d" corrispondono.

N.B) Puoi anche usare range: `[a-d]`.

3. `[^abcd]` → un singolo carattere NON tra a, b, c, d.

Esempio: "x" corrisponde, "a" no.

4. `.` → un singolo carattere qualsiasi.

Esempio: `c.t` trova "cat", "cut", "c9t".

5. `a*` → zero o più occorrenze di a.

Esempio: `a*` su "caaaane" → "aaaa".

6. a^+ → una o più occorrenze di a.

Esempio: "cane" → "a", "caaaaane" → "aaaa".

7. $a^?$ → al massimo una occorrenza di a (0 o 1).

Esempio: "cane" → "a", "cne" → ok.

8. $a\{N\}$ → esattamente N occorrenze di a.

Esempio: "aa" corrisponde a $a\{2\}$.

9. $a\{N,\}$ → almeno N occorrenze di a.

Esempio: "aaa" corrisponde a $a\{2,\}$.

10. $a\{,N\}$ → al più N occorrenze di a.

Esempio: $a\{,3\}$ accetta "", "a", "aa", "aaa".

11. $a\{N,M\}$ → tra N e M occorrenze di a.

Esempio: $a\{2,4\}$ corrisponde a "aa", "aaa", "aaaa".

12. a → cerca il carattere a all'inizio della riga.

Esempio: "ape" → matcha a (inizio riga) mentre "cane" → no.

13. $a\$$ → cerca il carattere a alla fine della riga.

Esempio: "panda" → matcha a finale mentre "ape" → no.

14. $\backslash <a$ → cerca a all'inizio di una parola.

Esempio: "ape carino" → matcha a di "ape" mentre "cane" → no (non è inizio parola).

15. `a\>` → cerca a alla fine di una parola.

Esempio: "casa bella" → matcha a di "casa" mentre "ape" → no (non è fine parola).

16. `regex1 | regex2` → OR logico tra due regex (matcha se vale una o l'altra).

Esempio: (cane|gatto) → trova "cane" oppure "gatto".

N.B) invece di un solo carattere (a, b, ecc.), possiamo mettere un gruppo di regex tra parentesi.

Quindi per gli esempi da 1-16 vale questa proprietà.

Esempio: 5. (abc)* → ripete il gruppo "abc" zero o più volte.

Classi di caratteri predefinite:

1. `[:upper:]` → lettere maiuscole (A-Z)

Esempio:

Matcha: A, Z

Non matcha: a, 1

2. `[:lower:]` → lettere minuscole (a-z)

Esempio:

Matcha: a, z

Non matcha: A, 5

3. `[:digit:]` → cifre numeriche (0-9)

Esempio:

Matcha: 3, 9

Non matcha: a, X

4. `[:alpha:]` → caratteri alfabetici (A-Z, a-z)

Esempio:

Matcha: b, K

Non matcha: 7, !

5. `[:alnum:]` → caratteri alfanumerici (A-Z, a-z, 0-9)

Esempio:

Matcha: c, 8

Non matcha: ?, .

6. `[:space:]` → ogni tipo di spazio (spazio, tab, newline, ecc.)

Esempio:

Matcha: " ", \t, \n

7. `[:blank:]` → spazi e tabulazioni (ma non newline)

Esempio:

Matcha: " ", \t

Non matcha: \n

8. `[:punct:]` → segni di punteggiatura (., ! ? ; : - ...)

Esempio:

Matcha: ,, !

Non matcha: a, 9

N.B) Queste classi vanno inserite dentro i selettori di classe [].

Esempio:

`[:upper:]` → matcha una sola lettera maiuscola.

`[:digit:]+` → matcha una o più cifre consecutive (es. "12345").

`[:alpha:]{3}` → matcha esattamente 3 lettere consecutive (es. "abc", "XYZ").

grep:

grep = Global Regular Expression Print → Mostra tutte le righe che corrispondono a una regex.

Sintassi: `grep [flags] pattern <percorso>`

Flag principali:

-E → usa espressioni regolari estese, quindi non serve il \ davanti a ?, +, {}, |, ().
(default: -G = regex basilari).

-c → conta quante righe corrispondono al pattern.

-n → mostra anche il numero di riga.

-v → mostra le righe che NON corrispondono al pattern.

-i → case-insensitive (ignora maiuscole/minuscole).

ESEMPI PRATICI:

Supponiamo di avere file.txt con il contenuto:

15 fifteen

14 fourteen

13 thirteen

12 twelve

11 eleven

10 ten

9 nine

8 eight

7 seven

6 six

5 five

4 four

3 three

2 two

1 one

1. `grep '[24]' file.txt` → seleziona tutte le righe che contengono 2 o 4:

14 fourteen
12 twelve
4 four
2 two

2. `grep 'ee' file.txt` → trova tutte le righe con "ee":

15 fifteen
14 fourteen
13 thirteen
11 eleven
3 three

3. `grep '^1.*e$' file.txt`

`^1` → inizio con 1
`.*` → qualsiasi sequenza
`e$` → finisce con e

12 twelve
1 one

4. `grep '\<[a-z]+n\>' file.txt`

`\<` → inizio parola
`[a-z]+` → sequenza di lettere
`n\>` → parola che finisce con n

15 fifteen
14 fourteen
10 ten

5. `grep -E '^1[04-9].*te+n$' file.txt`

`^1[04-9]` → righe che iniziano con 10, 14, 15, ...
`.*` → qualsiasi cosa in mezzo
`te+n$` → che finiscono con "ten", "teen", "teeeen"

15 fifteen
14 fourteen
10 ten

Find:

Cerca file all'interno della directory specificata e delle sue sottodirectory.

Supporta pattern semplici (*, ?, [abc]) o regex.

Sintassi: `find <percorso> [flags]`

<percorso> → directory di partenza (es. . per la corrente).

[flags] → opzioni di ricerca/filtraggio.

Flag principali:

`-name <pattern>` -> Cerca file con nome che corrisponde al pattern

`-regex <regex>` -> Cerca file con percorso che corrisponde a una regex.

N.B) Per regex estese: `-regextype posix-extended`.

`-user <nome> / -group <nome>` -> Filtra per proprietario o gruppo.

`-type <tipo>` -> Cerca solo un certo tipo di file:

f = file normali

d = directory

l = link simbolici

`-mindepth N / -maxdepth N`

Profondità minima/massima di ricerca nelle sottodirectory.

`-size [+|-]N[c/k/M/G]`

Filtra in base alla dimensione:

N = esattamente N unità

+N = maggiore di N

-N = minore di N

Differenza chiave con grep:

grep → cerca testo dentro file

find → cerca file o directory in base a nome, dimensione, tipo, permessi ecc.

ESEMPI:

1. `find . -name "*.txt"` -> Trova tutti i file con estensione .txt nella cartella corrente e sottocartelle.
2. `find . -regex ".*\.[ch]"` -> Trova tutti i file che finiscono con .c o .h (regex su percorso completo).
3. `find /home -user mario` -> Trova tutti i file di proprietà dell'utente mario.
4. `find . -type d` -> Trova tutte le directory.
5. `find . -maxdepth 1 -name "*.txt"` -> Cerca solo i .txt nella cartella corrente (non scende nelle sottocartelle).
6. `find . -size +10M` -> Trova file più grandi di 10 MB.

Sed:

Modifica, sostituisce o elimina testo mentre passa nello stream.

Non modifica i file originali a meno che non usi l'opzione -i.

Sintassi: `sed [flags] <percorso>`

Flag: `-e <script>` -> esegue lo script specificato (le istruzioni di sed).

Comandi principali:

- Eliminare righe che matchano una regex:

`sed -e '/regex/d' file.txt` -> Elimina (d) tutte le righe che contengono la regex.

- Eliminare righe tra due indici:

`sed -e '5,10d' file.txt` -> Elimina dalla riga 5 alla riga 10 incluse.

- Eliminare righe che NON matchano:

`sed -e '/regex/!d' file.txt` -> Tieni solo le righe che corrispondono alla regex, elimina le altre.

- Sostituire la prima occorrenza di un match:

`sed -e 's/cane/gatto/' file.txt` -> Sostituisce la prima occorrenza di "cane" con "gatto" su ogni riga.

- Sostituire tutte le occorrenze di un match:

`sed -e 's/cane/gatto/g' file.txt` -> Sostituisce **tutte** le occorrenze di "cane" con "gatto" in ogni riga.

ESEMPI:

Supponi che il file `animali.txt` contenga:

cane e cane

il gatto corre

cane dorme

1. `sed -e '/cane/d' animali.txt` → elimina tutte le righe che contengono "cane".

OUTPUT:

il gatto corre

2. `sed -e 's/cane/gatto/' animali.txt` → sostituisce solo la **prima occorrenza** di "cane" per riga.

OUTPUT:

gatto e cane

il gatto corre

gatto dorme

3. `sed -e 's/cane/gatto/g' animali.txt` → sostituisce **tutte le occorrenze** di "cane".

OUTPUT:

gatto e gatto

il gatto corre

gatto dorme

Utenti e gruppi:

Superutente (root):

- Ha tutti i privilegi sul sistema.
- Può creare/eliminare utenti, installare software, modificare permessi.
- Nome utente tipico: root, UID = 0.

Utenti regolari

- Non hanno privilegi amministrativi.
- Possono lavorare solo nei propri file e directory, a meno che non abbiano permessi speciali.
- UID diverso da 0.

Gruppi di utenti:

- Ogni utente appartiene sempre a un gruppo primario (associato automaticamente ai file creati).
- Può inoltre appartenere a più gruppi secondari, che servono per condividere permessi con altri utenti.

File /etc/group:

I gruppi sono memorizzati in /etc/group e hanno questa struttura:

group:password:gid:user1,user2,...,userN

- group → nome del gruppo
- password → di solito vuota o inutilizzata (campo obsoleto)
- gid → identificativo numerico del gruppo
- user1,...,userN → lista di utenti appartenenti al gruppo (secondari)

Esempio:

1. root:x:0 → gruppo root, GID=0, senza membri aggiuntivi.
2. users:x:100:alice,bob → gruppo users (GID=100) con membri alice e bob.

N.B) x => quel gruppo non ha una password associata

who, whoami, groups:

who -> Mostra gli utenti loggati sul sistema e informazioni su di loro.

Sintassi: `who [flags]`

Flag principali:

`-a` -> informazioni estese (equivalente a usare più flag insieme).

`-l` -> mostra i processi di login.

`-q` -> mostra solo i nomi degli utenti loggati e il numero totale.

`-u` -> mostra utenti loggati con idle time (da quanto non usano la shell) e PID della loro shell/sessione.

Whoami -> Mostra il nome dell'utente corrente (quello che stai usando nella shell).

Sintassi: `whoami`

Groups -> Mostra i gruppi a cui appartiene l'utente loggato o un utente specifico passato come argomento.

Sintassi: `groups [utente]`

ESEMPLI:

1. who -> Mostra chi è connesso al sistema.

Output tipico:

```
mario  tty1      2025-08-20 08:15
alice  pts/0     2025-08-20 09:01 (:0)
bob    pts/1     2025-08-20 09:30 (192.168.1.10)
```

mario loggato dalla console (tty1).

alice connessa da terminale grafico locale (:0).

bob loggato via SSH dall'IP 192.168.1.10.

2. whoami -> Mostra **il tuo utente corrente**.

Output tipico: mario

3. groups -> Mostra i gruppi a cui appartieni.

Output tipico:

mario : mario sudo users devs

utente mario appartiene ai gruppi:

mario (gruppo primario),

sudo (può usare sudo),

users (gruppo comune),

devs (sviluppatori).

id, last:

id -> Mostra l'identificativo numerico (UID/GID) dell'utente e i gruppi a cui appartiene.

Sintassi: **id [flags] [utente]**

Se non specifichi l'utente -> mostra info sull'utente corrente.

Flag principali:

-g -> mostra solo il GID (gruppo primario).

-G -> mostra tutti i GID dei gruppi a cui appartiene.

-u -> mostra solo l'UID.

-n -> mostra i nomi invece degli ID (va combinato con gli altri).

Esempio:

id

Output: uid=1000(mario) gid=1000(mario) groups=1000(mario),27(sudo),100(users)

Last -> Mostra gli ultimi login/logout degli utenti e gli ultimi riavvii del sistema, leggendo il file

Sintassi: **last**

Esempio:

last

Output tipico:

```
mario pts/0 :0      Wed Aug 20 09:15  still logged in
alice pts/1 192.168.1.20 Wed Aug 20 08:50 - 09:10 (00:20)
reboot system boot 5.15.0-79-generic Wed Aug 20 08:00  still running
```

Significa:

- mario è loggato ora su pts/0 (console grafica locale).
- alice si è collegata via SSH da 192.168.1.20 ed è rimasta loggata 20 minuti.
- reboot indica un riavvio avvenuto alle 08:00.

su, sudo:

su (substitute user) -> Serve per cambiare utente.

Se non specifichi un utente -> di default diventi **root**.

Ti chiede la password dell'utente in cui vuoi entrare.

Rimani loggato come quell'utente finché non fai exit.

Sintassi: **su [utente]**

ESEMPIO: su alice -> Cambia utente in alice (password di alice richiesta).

sudo (superuser do) -> Serve per eseguire un comando come un altro utente (di default root).

Non cambi utente, resti nel tuo → il comando viene eseguito con privilegi elevati.

Richiede la tua password (se sei abilitato in /etc/sudoers).

Sintassi: **sudo [flags] comando**

Flag utili:

-H → usa la home directory dell'utente corrente.

-u user → esegue il comando come l'utente specificato (non per forza root).

ESEMPIO: sudo -u alice whoami -> Esegue whoami come utente alice.

Differenza chiave:

su → cambi utente (apri una sessione nuova).

sudo → esegui un singolo comando come un altro utente.

date:

Mostra la data e ora correnti.

Può anche modificare la data/ora (solo l'utente root può farlo).

Supporta formati human-readable.

Sintassi: **date [flags] [data]**

Flag principali

-d <data> -> Mostra la data e ora a partire da una stringa fornita.

-r <file> -> Mostra la data/ora dell'ultima modifica del file.

-s <data> -> Imposta la data/ora del sistema a quella specificata (solo root).

ESEMPI:

1. date

Output:

Wed Aug 20 19:30:45 CEST 2025

2. date -d "2025-08-20 10:30"

Output:

Wed Aug 20 10:30:00 CEST 2025

3. date -r documento.txt

Output:

Tue Aug 19 22:14:05 CEST 2025

4. sudo date -s "2025-12-31 23:45"

Output:

Wed Dec 31 23:45:00 CET 2025

Shutdown:

Spegne o riavvia il sistema.

Può essere eseguito solo da root (o con sudo).

Invia un messaggio di avviso a tutti gli utenti loggati.

Sintassi: `shutdown [flags] <time> [message]`

Flag principali

`-k` → non spegne, manda solo un messaggio di avviso.

`-r` → riavvia il sistema.

`-h` → spegne il sistema (halt).

`-t <N>` → spegne il sistema dopo N secondi.

Formati di <time>:

hh:mm → spegne all'orario specificato.

+N → spegne dopo N minuti.

now → spegne subito.

ESEMPIO: `sudo shutdown -h +10 "Il sistema si spegnerà tra 10 minuti"`

Passwd:

Modifica la password dell'utente loggato (o di un altro utente, se sei root).

Flusso tipico:

Ti chiede la password attuale.

Ti chiede la nuova password.

Ne controlla la complessità (lunghezza, numeri, caratteri speciali, ecc.).

Esempio: **passwd** -> Cambia la tua password.

Il file /etc/passwd:

Contiene informazioni sugli account degli utenti.

Ogni riga rappresenta un utente con questa struttura:

user:password:uid:gid:info:home:shell

Significato dei campi:

user → nome utente (es. alice)

password → password cifrata (storicamente qui, oggi di solito x → vuol dire che la password sta in /etc/shadow)

uid → identificativo numerico dell'utente (0=root, 1000+=utenti normali)

gid → identificativo numerico del gruppo principale dell'utente

info → informazioni aggiuntive (campo GECOS: nome reale, telefono, ecc.)

home → directory personale dell'utente (es. /home/alice)

shell → shell di login di default (es. /bin/bash, /bin/zsh, /usr/sbin/nologin per utenti di servizio)

ESEMPIO:

alice:x:1000:1000:Alice Rossi,,,:/home/alice:/bin/bash

alice → utente normale (UID=1000), gruppo primario 1000, home /home/alice, shell /bin/bash.

/etc/shadow:

Contiene gli hash delle password degli utenti (non le password in chiaro).

Accessibile solo da root per motivi di sicurezza.

Ogni riga rappresenta un utente e ha la struttura:

user:password:last_change:min:max:warn:inactive:expire

Campi principali:

user → nome utente.

password → hash della password.

- Se vuoto → non serve password.
- Se contiene valori speciali:
 - * o ! → login disabilitato.
 - !! → password non inizializzata.

last_change → ultimo cambio password (in giorni dall'epoch UNIX: 1/1/1970).

min → minimo numero di giorni che devono passare dall'ultimo cambio password prima di poterla cambiare di nuovo.

max → massimo numero di giorni per cui la password rimane valida.

warn → numero di giorni di preavviso prima che la password scada.

inactive → numero di giorni dopo la scadenza della password oltre i quali l'account viene disabilitato.

expire → data di scadenza dell'account (dopo non si può più loggare).

ESEMPIO:

alice:\$6\$Ghs82Ks\$9A1e...:19520:7:90:14:7:20000:

alice → password cifrata, cambiata 19520 giorni dopo epoch, può cambiarla ogni 7 giorni, scade dopo 90 giorni, avviso 14 giorni prima, account scade al giorno 20000.

Useradd:

Crea un nuovo utente.

Può anche aggiornare i parametri di default per i futuri utenti.

Sintassi: **useradd [flags] [user | -D]**

Flag principali:

-D → non crea un utente, ma aggiorna le informazioni di default.

-d <home_dir> → imposta la directory home dell'utente.

-m → crea la directory home se non esiste.

-g <group> → imposta il gruppo principale (deve esistere).

-G <group1,group2,...> → aggiunge l'utente a gruppi secondari (devono esistere).

-p <password> → imposta la password cifrata (hash, generata con crypt).

-s <shell> → specifica la shell di login (/bin/bash, /bin/zsh, ecc.).

-u <uid> → assegna un UID personalizzato.

ESEMPIO:

1. `sudo useradd -m -s /bin/zsh mario` -> Crea utente mario con shell /bin/zsh.
2. `sudo useradd -m -g users -G sudo,devs mario` -> Crea utente mario con gruppo primario users e secondari sudo, devs.
3. `sudo useradd -m -d /srv/mario mario` -> Crea utente mario con home /srv/mario.
4. `sudo useradd -m mario` -> Crea utente mario con home /home/mario e shell di default.

userdel, groupadd, groupdel:

userdel -> Rimuove un account utente.

Non sempre elimina i file associati all'utente (home, mail, ecc.), a meno che non venga usato un flag.

Sintassi: **userdel [flags] <user>**

Flag: **-r** → rimuove anche i file nella home directory dell'utente.

ESEMPIO:

sudo userdel -r mario -> Rimuove l'utente mario e la sua home directory /home/mario.

groupadd -> Crea un nuovo gruppo.

Sintassi: **groupadd [flags] <group>**

Flag: **-g** <gid> → specifica l'ID numerico del gruppo.

ESEMPIO:

sudo groupadd devs -> Crea il gruppo devs con un GID scelto automaticamente.

Groupdel -> Rimuove un gruppo.

N.B) Non è possibile eliminare un gruppo se è il gruppo primario di un utente esistente.

Sintassi: **groupdel <group>**

ESEMPIO:

sudo groupdel devs -> Elimina il gruppo devs (solo se non è gruppo primario di nessun utente).

Archiviazione e filesystem:

In Linux, i dischi e le periferiche non compaiono come “unità” (C:, D: come in Windows), ma come file speciali a blocchi dentro la directory /dev.

Struttura dei nomi

- Le prime due lettere → tipo di device / canale di connessione.
- La terza lettera → numero del device sul canale.
- La quarta cifra → numero della partizione.

Esempi:

/dev/hda → primo hard disk o unità ottica su canale PATA (master).

/dev/hdd3 → terza partizione del disco slave sul secondo canale PATA.

/dev/sdb1 → prima partizione del secondo hard disk SATA/SCSI/USB.

Montaggio dei filesystem:

Per accedere ai file dentro un disco/partizione, il suo filesystem deve essere montato dentro la gerarchia globale che ha radice in /.

- Anche la partizione principale del sistema operativo (root /) viene montata automaticamente all'avvio.
- Le periferiche esterne (USB, CD, dischi esterni) di solito vengono montate automaticamente in:
 - /mnt → mount point generico.
 - /media → mount point usato da molte distro moderne.

Esempio pratico:

sudo mount /dev/sdb1 /mnt/usb -> Monta la prima partizione del disco /dev/sdb nella cartella /mnt/usb.

mtab, fstab:

/etc/mtab:

- Contiene l'elenco di tutti i mount attualmente attivi.
- Include anche le opzioni di montaggio usate.
- Viene aggiornato dinamicamente dal sistema man mano che si montano/smontano filesystem.

È quindi una fotografia dello stato corrente dei mount.

/etc/fstab:

- Contiene i mount point statici, cioè quelli che devono essere montati automaticamente all'avvio.
- Permette di specificare come e dove montare ogni device.
- Se si aggiunge l'opzione noauto, il device non viene montato automaticamente.

Serve per la configurazione permanente del montaggio.

Struttura delle righe in fstab (e mtab):

device mount_point fs_type options dump passno

Campi:

device → nome del file associato al dispositivo (es. /dev/sda1, oppure UUID).

mount_point → directory in cui montare il filesystem (es. /, /home, /mnt/usb).

fs_type → tipo di filesystem (ext4, ntfs, vfat, xfs, ecc.).

options → opzioni di montaggio (es. rw, ro, noexec, separate da virgola).

dump → usato dai programmi di backup (0=non fare backup, 1=abilitato).

passno → ordine con cui i filesystem vengono controllati da fsck all'avvio:

0 → nessun controllo.

1 → controllato per primo (/).

2 → controllati dopo (/home, /var, ecc.).

Esempio di /etc/fstab:

```
UUID=1234-ABCD / ext4 defaults 1 1
```

```
UUID=5678-EFGH /home ext4 defaults 0 2
```

```
/dev/sdb1 /mnt/usb vfat noauto,user,rw 0 0
```

Spiegazione:

- La root / (ext4) viene montata all'avvio, con controllo fsck prioritario (1).
- La /home (ext4) viene montata con controllo successivo (2).
- La chiavetta USB /dev/sdb1 (vfat) può essere montata manualmente dall'utente (noauto,user), non viene controllata da fsck (0).

mount, umount:

mount:

Monta un filesystem (collega un device a una directory).

Se esiste un mount point in /etc/fstab, basta specificare il nome del device o la directory.

/etc/fstab → contiene i mount permanenti.

Senza argomenti → mostra i dispositivi attualmente montati.

Sintassi: **mount [flags] [device] [directory]**

Flag principali:

-a → monta tutti i filesystem specificati in /etc/fstab.

-o <options> → specifica le opzioni di montaggio:

ro → sola lettura (read-only).

rw → lettura e scrittura (read-write).

noauto → non viene montato automaticamente.

-t <type> → specifica il tipo di filesystem (ext4, vfat, ntfs, ecc.).

Se omesso o auto → il sistema tenta di riconoscerlo.

ESEMPIO: `sudo mount /dev/sdb1 /mnt/usb` -> Montare una chiavetta USB in /mnt/usb

umount:

Smonta un filesystem → rende il device inaccessibile fino al prossimo mount.

Sintassi: `umount [device | mount_point]`

ESEMPIO: `sudo umount /mnt/usb` -> Smonta la chiavetta montata in /mnt/usb.

df:

Mostra informazioni sull'**utilizzo dei filesystem** (spazio totale, spazio usato, spazio disponibile)

Senza argomenti → mostra tutti i filesystem attualmente montati.

Sintassi: `df [flags] [file] ...`

Flag principali:

`-i` → mostra informazioni in termini di inode (quanti inode sono usati/liberi).

Utile quando non è lo spazio a disco a mancare, ma il numero massimo di file (inode) disponibili.

ESEMPIO: `df ~/.bashrc` -> Mostra info sul filesystem dove si trova il file ~/.bashrc (cioè /home).

du:

Mostra lo spazio su disco utilizzato dai file o directory specificati.

Con le directory → calcola ricorsivamente anche i contenuti.

Senza argomenti → mostra lo spazio della directory corrente.

Sintassi: `du [flags] [file]...`

Flag principali:

-a → mostra anche lo spazio occupato da singoli file, non solo directory.

-B N o --block-size=N → dimensione dei blocchi in N byte (default 1024 → output in KB).

-d N o --max-depth=N → limita la profondità della ricorsione (es. -d 1 → mostra solo directory di primo livello).

ESEMPIO: du

Output tipico (in KB):

12 ./Documenti

8 ./Immagini

24 .

Gestione dei Job in Linux:

Un job = ogni pipeline di comandi avviata nella shell.

Esecuzione:

Foreground (default): La shell aspetta che il job termini → esecuzione sincrona.

Esempio: sleep 10 (la shell rimane bloccata per 10 secondi)

Background: Se aggiungi & alla fine → la shell non aspetta → esecuzione asincrona.

Esempio: sleep 10 & (Restituisce subito il controllo mostrando [job number] PID)

Informazioni mostrate:

Quando avvii un job in background, la shell mostra:

Numero del job → jobspec (es. [1])

PID → ID di processo (es. 1234)

ESEMPIO: gedit &

Output: [1] 4321

#job n. 1, PID = 4321)

Scorciatoie da tastiera:

CTRL+Z: sospende il job in foreground, lo sposta in background e restituisce il controllo alla shell.

CTRL+C: termina il job in foreground.

sleep, yes:

sleep: sospende l'esecuzione per un certo numero di secondi.

Sintassi: **sleep <N>** dove <N> è il numero di secondi.

ESEMPLI:

sleep 5 # sospende per 5 secondi

sleep 2m # sospende per 2 minuti

sleep 1h # sospende per 1 ora

yes: stampa una stringa ripetutamente fino all'interruzione (CTRL+C).

Sintassi: **yes [stringa]**

ESEMPLI:

yes # stampa "y" all'infinito

yes ciao # stampa "ciao" all'infinito

yes "Linux è bello!" # ripete la frase all'infinito

fg, bg, jobs:

fg (foreground): Porta in foreground un job (cioè sotto controllo diretto della shell).

Se non si specifica nulla, riprende l'ultimo job referenziato.

Sintassi: **fg [jobspec]**

ESEMPIO: fg %1 # porta in foreground il job numero 1

bg (background): Riprende in background un job sospeso.

Anche qui, se non si specifica nulla, agisce sull'ultimo job.

Sintassi: `bg [jobspec]`

Esempio: `bg %2` # riprende in background il job numero 2

Jobs: Mostra lo stato dei job avviati nella sessione corrente.

Può mostrare informazioni estese o solo i PID.

Sintassi: `jobs [flags] [jobspec]`

Flag principali:

-l → visualizzazione estesa (incluso il PID).

-p → mostra solo i PID.

ESEMPLI:

`jobs -l` # mostra lo stato dei job con i PID

`jobs -p` # mostra solo i PID dei job

Scenario tipico di utilizzo:

Lanci un comando lungo in foreground → `tar -czf backup.tar.gz /home`

Lo sospendi → CTRL+Z

Lo riprendi in background → `bg`

Controlli lo stato → `jobs`

Se vuoi riportarlo in foreground → `fg %1`

ps:

ps serve per visualizzare i processi attivi sul sistema.

Sintassi: `ps [flags]`

Flag (stile UNIX)

`-u [utenti]` → mostra solo i processi appartenenti a determinati utenti.

`-p [pid]` → mostra solo i processi con il PID specificato.

`-f` → visualizzazione estesa (mostra PPID, UID, percorso comando, ecc.).

Flag (stile BSD)

`r` → mostra solo i processi in esecuzione.

`a` → mostra anche i processi non appartenenti all'utente corrente (esclusi quelli senza terminale).

`x` → mostra anche i processi senza terminale.

`u` → output in formato "user-oriented" (utente, CPU, memoria, ecc.).

top:

top mostra in **tempo reale** i processi attivi.

Aggiorna continuamente i dati (di default ogni 3 secondi).

Permette di ordinare i processi per CPU, memoria o altri criteri.

Sintassi: `top`

Struttura dell'output:

1. Parte alta (informazioni generali sul sistema):

- Load average: carico medio della CPU negli ultimi 1, 5 e 15 minuti.
- Task: numero totale di processi e quanti sono in esecuzione, sospesi, zombie.
- CPU%: percentuale di utilizzo CPU (user, system, idle, ecc.).
- Memoria RAM e SWAP: totale, usata, libera, buffer/cache.

2. Parte bassa (tabella processi):

- PID: ID processo.
- USER: utente proprietario.
- %CPU: percentuale di CPU utilizzata.
- %MEM: percentuale di memoria utilizzata.
- TIME+: tempo CPU totale usato.
- COMMAND: comando avviato.

Interazione durante l'esecuzione:

q → esci.

k → termina un processo (chiede PID).

h → help.

Shift+P → ordina per uso CPU.

Shift+M → ordina per uso memoria.

Shift+F → cambia l'ordinamento interattivamente.

Differenza ps vs top:

ps → fornisce un'istantanea dei processi attivi in quel momento (non cambia).

top → mostra una vista dinamica e aggiornata in tempo reale.

ESEMPIO:

```
top - 14:46:22 up 1:40, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 115 total, 1 running, 114 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem:  973.8 total,  746.0 free,  93.0 used,  134.8 buff/cache
MiB Swap:  950.0 total,  950.0 free,  0.0 used,  806.6 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
769	ivano	20	0	9848	3304	2692	R	0.3	0.3	0:00.07	top
1	root	20	0	164528	9372	7128	S	0.0	0.9	0:00.61	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par_gp
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	worker/0:0H-events_highpri
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_percpu_wq
9	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_tasks_rude
10	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_tasks_trace
11	root	20	0	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/0
12	root	20	0	0	0	0	I	0.0	0.0	0:00.17	rcu_sched
13	root	rt	0	0	0	0	S	0.0	0.0	0:00.17	migration/0
14	root	20	0	0	0	0	I	0.0	0.0	0:00.11	worker/0:1-events
15	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
16	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
17	root	rt	0	0	0	0	S	0.0	0.0	0:00.17	migration/1
18	root	20	0	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/1
20	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	worker/1:0H-events_highpri
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/2
22	root	rt	0	0	0	0	S	0.0	0.0	0:00.18	migration/2
23	root	20	0	0	0	0	S	0.0	0.0	0:00.05	ksoftirqd/2
25	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	worker/2:0H-events_highpri
26	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/3
27	root	rt	0	0	0	0	S	0.0	0.0	0:00.20	migration/3
28	root	20	0	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/3
30	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	worker/3:0H-events_highpri
31	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/4
32	root	rt	0	0	0	0	S	0.0	0.0	0:00.16	migration/4

kill, killall:

kill: invia un segnale ad un processo tramite il suo PID (Process ID).

Sintassi: `kill [flags] <PID>`

Segnale di default: SIGTERM (15) → chiede al processo di terminare in modo pulito.

Flag principali:

`-s <segnale>` oppure `-SEGNALENAME` → invia un segnale specifico.

ESEMPIO: `kill -s SIGKILL 1234` → forza la chiusura del processo con PID 1234.

Killall: invia segnali a tutti i processi che hanno un certo nome.

Sintassi: `killall [flags] <nome>`

Flag principali:

`-s <segnale>` → specifica il segnale (come in kill).

`-u <utente>` → limita l'azione ai processi di un determinato utente.

`-i` → chiede conferma prima di uccidere ogni processo

Differenza kill vs killall:

kill → agisce su un singolo processo identificato dal suo PID.

killall → agisce su tutti i processi con un dato nome.

Segnali:

Numero	Nome segnale	Descrizione	Tipico utilizzo
1	SIGHUP	Disconnessione del terminale (hang up).	Spesso usato per ricaricare la configurazione di un demone senza riavviarlo.
2	SIGINT	Interruzione (CTRL+C).	Interrompe un processo in foreground.
3	SIGQUIT	Terminazione con <i>core dump</i> .	Debug di errori critici.
6	SIGABRT	Terminazione per errore critico (<i>abort</i>).	Generato dalle librerie in caso di errori gravi.
9	SIGKILL	Terminazione forzata, non può essere ignorato né gestito.	Uccidere un processo bloccato.
11	SIGSEGV	Errore di segmentazione (accesso a memoria non valida).	Indica bug nel programma.
15	SIGTERM	Richiesta di terminazione "gentile".	Default di kill, il processo può gestirlo e chiudersi correttamente.
19	SIGSTOP	Sospensione del processo (CTRL+Z).	Congela il processo senza terminarlo.

Differenze chiave:

SIGTERM (15): terminazione pulita → il processo può salvare dati e liberare risorse.

SIGKILL (9): terminazione immediata e forzata → il processo viene rimosso dal kernel senza possibilità di pulizia.

SIGSTOP (19): sospende un processo (può essere riattivato con SIGCONT).

ESEMPI:

1. `kill -SIGTERM 1234` → Terminare in modo pulito un processo (PID 1234)
2. `kill -SIGKILL 1234` → Forzare la chiusura (quando non risponde)
3. `kill -SIGSTOP 1234` → Sospendere un processo
4. `kill -SIGCONT 1234` → Riprendere un processo sospeso